BÀI KIỂM TRA GIỮA KỲ

Quản lý món ăn

Nguyễn Đức Khiêm - BIT240125
Môn: Phát triển ứng dụng di động
Project: **BIT240125_FoodManager**
Bản bàn giao ngày 20/10/2026

Android Native Java + SQLite + MockAPI

Ứng dụng giao diện hồng, mặc định English và chuyển được sang Tiếng Việt. Bài có 3 loại, 6 món mẫu, quản lý nhiều ảnh, lưu offline và phần tự đồng bộ với dịch vụ MockAPI thật. Đề không yêu cầu một màu giao diện cụ thể, nên lựa chọn màu hồng không làm thay đổi tiêu chí chức năng.

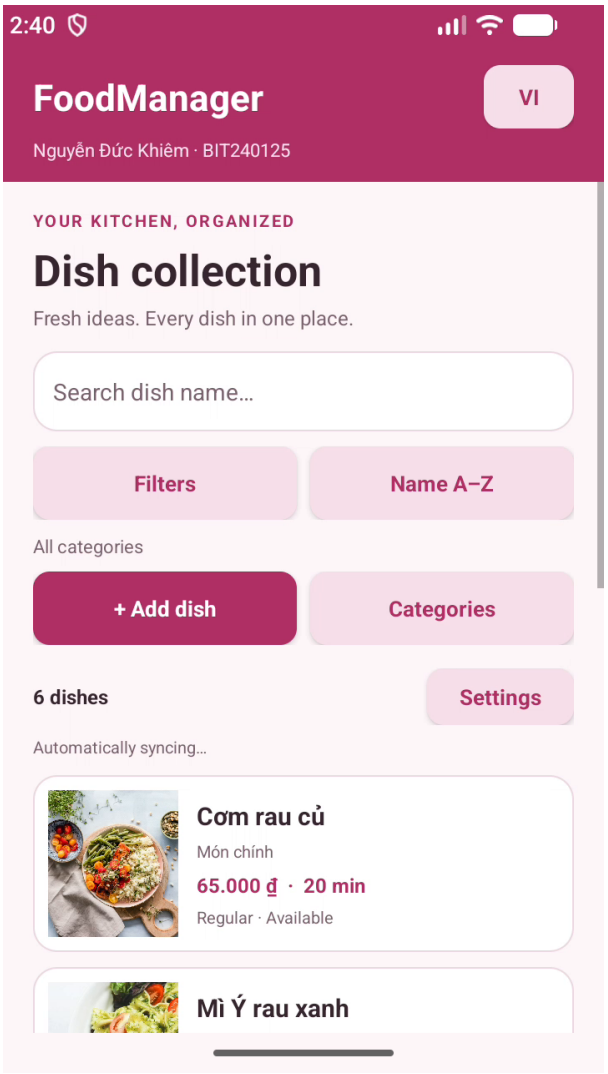
Đối chiếu các nhóm yêu cầu

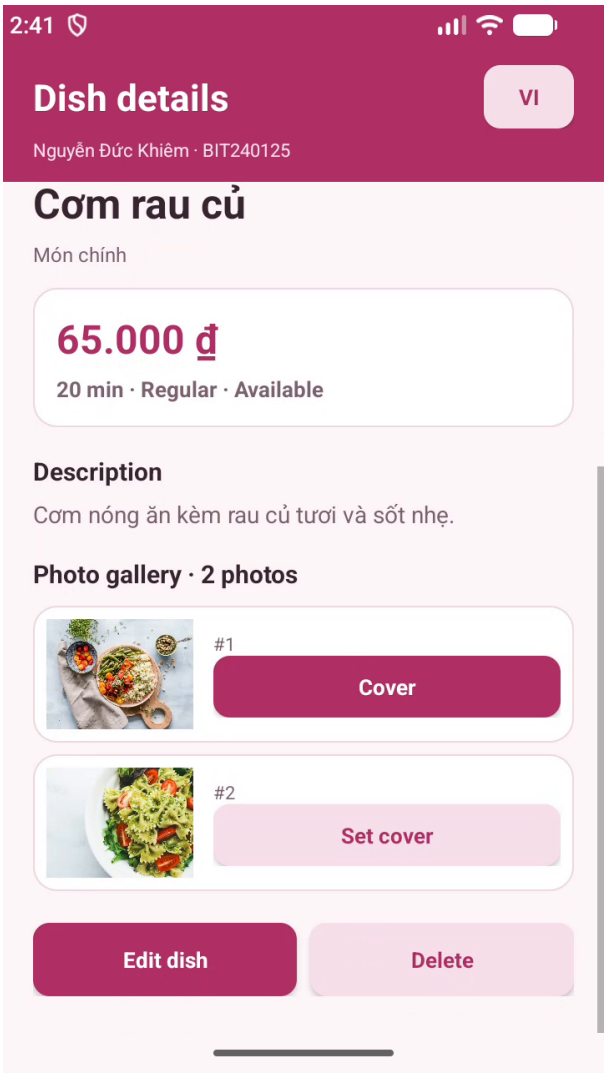
Nhóm yêu cầu	Triển khai / minh chứng
Model, dữ liệu và validation	Đủ thuộc tính; tiền tố BIT240125_; 3 loại/6 món; kiểm tra cả nghiệp vụ và SQLite.
CRUD và giao diện	Danh sách, chi tiết, thêm/sửa/xóa có xác nhận; quản lý loại món.
Tìm kiếm và bộ lọc	Tìm một phần tên; kết hợp loại, trạng thái, giá; sắp xếp; giữ bộ lọc.
Ảnh và thời gian	Gallery nhiều ảnh, đổi đại diện; tối đa một cover; Quick khi <= 15 phút.
Đa ngôn ngữ	English mặc định; dịch nhãn/nút/trạng thái/lỗi; giữ lựa chọn.
Offline và tự đồng bộ	Ghi SQLite + outbox; phát hiện mạng; REST upsert với UUID; kiểm tra API thật.
Video bắt buộc	Video 1 quay emulator; Video 2 có lời đọc tổng hợp tiếng Việt và source thực tế.

Gói bàn giao gồm APK, mã nguồn có Gradle wrapper, hai video, README chi tiết, schema API và log/JSON kiểm chứng. Bạn đã chọn tự đưa mã nguồn lên GitHub sau. Chưa có thao tác nộp LMS thay bạn.

1. Giao diện và cách dùng

Giao diện sử dụng hồng đậm cho tiêu đề và hành động chính, nền hồng nhạt và thẻ trắng. Chữ và trạng thái có độ tương phản rõ. Các ảnh dưới đây được trích từ bản chạy thực tế trên emulator.





Danh sách	Chi tiết
Search tìm một phần tên. Filters kết hợp điều kiện. Nút sắp xếp cho giá tăng/giảm và thời gian tăng. Add dish mở form; Categories quản lý loại.	Xem giá, trạng thái, mô tả và ảnh. Set cover đổi ảnh đại diện. Edit dish chỉnh sửa; Delete luôn mở xác nhận.

Nút VI/EN đặt trên đầu mỗi màn hình. Không có yêu cầu dịch dữ liệu tên/mô tả món do người dùng nhập; vì vậy tên món tiếng Việt vẫn giữ nguyên khi giao diện English.

2. Model, schema và validation

Quan hệ dữ liệu

```
BIT240125_DishCategory (1) ---- (N) BIT240125_Dish (1) ---- (N) BIT240125_DishImage
```

Bảng	Các cột (đều có tiền tố BIT240125_)
DishCategory	Id, Name, Description
Dish	Id, Name, NameKey, Price, PreparationTime, IsAvailable, Description, DishCategoryId
DishImage	Id, DishId, ImageUrl, IsThumbnail
Outbox	Entity, EntityId, Operation, Revision

Tên viết rút gọn trong bảng trên để dễ đọc. Tên SQL đầy đủ, ví dụ: BIT240125_Dish, BIT240125_Price. Contract.java ghép toàn bộ tên từ hằng số P. NameKey và Outbox là dữ liệu phụ trợ để bảo đảm duy nhất và đồng bộ. SQLite/Android có thể tạo metadata nội bộ; các bảng nghiệp vụ do ứng dụng định nghĩa đều theo quy tắc MSSV.

Các ràng buộc chính

• **Name:** trim, gom khoảng trắng, chuẩn hóa Unicode NFC và chuyển chữ thường để so trùng. Tên không rỗng. UNIQUE(DishCategoryId, NameKey) chặn trùng trong cùng loại.

• **Price:** số nguyên dương theo VND. **PreparationTime:** số nguyên dương theo phút. Bộ lọc giá cho phép 0 nhưng min không được vượt max.

• **DishCategoryId:** phải có trong dữ liệu lưu trữ. Bật foreign key; loại đang có món bị ON DELETE RESTRICT. Xóa món dùng ON DELETE CASCADE để xóa ảnh.

• **Ảnh:** URL HTTPS hợp lệ, không trùng trong một món. Partial UNIQUE index trên DishId khi IsThumbnail = 1 chặn hai cover. Tối đa một ảnh đại diện; cho phép danh sách ảnh rỗng.

• **Thời gian:** <= 15 phút là Quick/Nhanh, còn lại Regular/Thông thường. Đây là thuộc tính suy ra, không lưu dư vào database.

Luồng ghi và đọc

Form → captureDraft → validate → transaction SQLite (món + ảnh + outbox) → đọc lại chi tiết. Xóa cần xác nhận và kiểm tra tồn tại/ràng buộc. Đọc danh sách áp dụng các điều kiện đã chọn và thứ tự sắp xếp. Món ngừng bán vẫn có ở danh sách quản lý chung, chỉ bị loại khi lọc đang bán.

ValidationException chứa khóa lỗi. MainActivity dùng Texts để hiển thị đúng ngôn ngữ. Id không tồn tại có thông báo; danh sách không kết quả có nút xóa lọc; ảnh lỗi có hình thay thế.

3. Offline và tự đồng bộ MockAPI

SQLite là nơi ứng dụng luôn đọc và ghi. Khi mất mạng, thay đổi được lưu ngay; không phụ thuộc việc gọi API thành công. SyncEngine theo dõi mạng và thử lại tự động trong lúc ứng dụng mở, không có nút yêu cầu người dùng đồng bộ.

Dịch vụ thật đã cấu hình

Project: https://mockapi.io/projects/6ad71854f71f154ac487c193

Loại món: https://6ad71854f71f154ac487c192.mockapi.io/api/v1/BIT240125_DishCategory

Món ăn: https://6ad71854f71f154ac487c192.mockapi.io/api/v1/BIT240125_Dish

SyncConfig.java chứa URL mặc định. Settings cho thay cả hai URL hoặc xóa cả hai để dùng local. MockAPI có trường id do dịch vụ tạo để định tuyến PUT/DELETE; mã nghiệp vụ vẫn là BIT240125_Id. Mọi trường tự định nghĩa và trường con trong mảng ảnh đều có tiền tố MSSV.

Thuật toán thực hiện

1. Trong cùng transaction lưu dữ liệu, ghi một mục outbox cho mỗi entity; mỗi lần thay đổi có Revision mới.

2. NetworkCallback kiểm tra Internet đã được hệ điều hành xác nhận. Khi có mạng, chạy worker đơn luồng; có thử lại định kỳ 30 giây.

3. GET dữ liệu để ánh xạ UUID local sang id dịch vụ. Đẩy xóa món trước, loại trước món khi upsert, xóa loại sau.

4. Nếu UUID đã có thì PUT; chỉ POST khi chưa có. Nếu mất phản hồi sau POST, lần thử lại GET thấy UUID nên không tạo trùng.

5. Chỉ xóa outbox có đúng Revision đã gửi. Sửa mới phát sinh trong lúc sync vẫn còn để gửi sau.

6. GET mới sau khi đẩy và nhập dữ liệu remote hợp lệ; bỏ qua bản ghi local còn pending để bảo vệ dữ liệu chưa gửi.

Kiểm chứng trong Video 1

Tắt Wi-Fi/dữ liệu emulator → tạo Mon offline BIT240125 v2 → sửa thành 40.000 → xác nhận API chưa có bản ghi → bật mạng → ứng dụng tự đồng bộ. Log và dữ liệu trước/sau xác nhận đúng một UUID trên MockAPI và giá 40.000. Không dùng nút đồng bộ.

Phạm vi: một thiết bị, ưu tiên local đang pending; worker chạy khi ứng dụng mở. Chưa triển khai xử lý xung đột nhiều thiết bị hoặc truyền thao tác xóa trực tiếp từ website về mọi thiết bị. Luồng xóa từ ứng dụng có tombstone và được đẩy lên API.

4. Cài đặt, kiểm thử và video

Mở và biên dịch

Giải nén mã nguồn, mở BIT240125_FoodManager trong Android Studio. Cài SDK Platform 34 và chọn Gradle JDK phù hợp. Bản đã kiểm tra dùng JBR 25, Gradle 9.6.0, Android Gradle Plugin 9.4.1. Lần đầu trên máy khác cần mạng để tải công cụ. Android Studio sẽ tạo local.properties theo SDK của máy.

```
.\gradlew.bat assembleDebug
# APK: app/build/outputs/apk/debug/app-debug.apk
```

Cài BIT240125_FoodManager.apk để chạy nhanh trên điện thoại/emulator Android 8.0 trở lên. Giao diện cài mới là English. MockAPI đã cấu hình sẵn; nếu đang mất mạng, dữ liệu vẫn lưu và sẽ gửi khi mở ứng dụng có mạng lại.

Kết quả kiểm thử

Nhóm	Minh chứng
67 kiểm tra dữ liệu	Schema/tên bảng/cột; foreign keys; CRUD; persistence; validation; lọc kết hợp; 3 cách sort; ảnh; outbox revision.
11 kiểm tra giao diện	Mặc định English; đổi và giữ ngôn ngữ; form; lỗi trùng tên/giá/thời gian/khoảng giá; Id không tồn tại.
Đồng bộ dịch vụ thật	Đã tự đưa 3 loại/6 món lên API. Có kiểm chứng offline trước/sau và số bản ghi theo UUID.
Video thao tác	Kịch bản chạy trên emulator, có xác nhận lưu/xóa, giữ bộ lọc và bản ghi remote.

Số 67 tính riêng từng cột khi kiểm tra tiền tố, không phải 67 chức năng. Test runner nằm trong androidTest, không đóng gói trong APK người dùng. Chế độ test dữ liệu giả định 3 loại/6 món nền, nên chạy trên emulator mới nếu đã có dữ liệu cần giữ.

Nội dung hai video

Video 1: màn hình emulator thực tế, giao diện hồng, danh sách/chi tiết, CRUD, lọc kết hợp, sắp xếp, ảnh, validation, loại món và chuyển ngôn ngữ; có đoạn offline và tự đồng bộ MockAPI.

Video 2: khoảng 7 phút, giọng đọc tổng hợp tiếng Việt. Các khung hình trích source thực tế có số dòng, chỉ rõ tên project, Contract, model, SQLite, validation, truy vấn, ảnh, đa ngôn ngữ và thuật toán sync. Đây là thuyết minh từ đoạn mã, không phải giả lập màn hình IDE.

5. Tự trình bày và nộp bài

Bản đồ mã nguồn cần nắm

File	Vai trò
settings.gradle	rootProject.name = BIT240125_FoodManager
Contract.java	Tiền tố MSSV và toàn bộ tên bảng/cột
FoodRepository.java	Schema, seed, CRUD, validate, query, giao dịch, outbox và import remote
MainActivity.java	Danh sách/chi tiết/form, xác nhận xóa, bộ lọc và ngôn ngữ
Texts.java / ImageLoader.java	Bản dịch và tải/cache ảnh
SyncConfig.java / SyncEngine.java	Endpoint thật; phát hiện mạng; ánh xạ UUID; POST/PUT/DELETE và retry

Các câu hỏi nên trả lời được

Tại sao tên trùng chỉ bị chặn trong cùng loại?
Ràng buộc UNIQUE gồm cặp CategoryId và NameKey.

Vì sao không xóa được loại đang dùng?
Kiểm tra COUNT ở repository và foreign key RESTRICT ở SQLite.

Làm sao bảo đảm một ảnh đại diện?
Bỏ cờ ở ảnh còn lại; partial unique index chặn vi phạm ở database.

Tại sao offline vẫn lưu được?
SQLite là nguồn dữ liệu chính; API là bước đồng bộ sau.

Vì sao retry không tạo trùng?
UUID giữ nguyên; worker tra UUID remote trước khi POST.

Vì sao sửa khi đang đồng bộ không bị mất?
Chỉ acknowledge đúng Revision cũ; Revision mới được giữ lại.

Các bước nộp theo đề

• Bạn tự tạo repository GitHub, tải source lên và lấy link Git. Giữ nguyên wrapper, source và assets; bỏ build, .gradle, .idea, local.properties như .gitignore.

• **Trong ca học kiểm tra:** nộp link mã nguồn và Video 1 lên LMS theo hướng dẫn lớp.

• **Chậm nhất ngày 23/10/2026:** nộp Video 2 lên LMS.

Đề cho phép công cụ AI hỗ trợ nhưng yêu cầu sinh viên hiểu, giải thích được mã nguồn và có thể bị yêu cầu sửa trực tiếp. Hãy mở README chi tiết và chạy thử trước khi demo. Các video có giọng tổng hợp được ghi rõ; phần thông tin cá nhân chỉ xuất hiện trong hình ảnh, không gửi tới dịch vụ giọng đọc.

Tài liệu đối chiếu: đề PDF do người dùng cung cấp (2 trang); mã nguồn, log kiểm thử và dữ liệu API của chính project này.